

ML 파이프라인 공부노트

4조 · LoL 승패 예측·설명 서비스 | 2026-08-31, Day 3 모델 선정 + 피처 재정리(diff13)까지 실측 반영

발표 주제: LoL 경기는 언제 승부가 결정되는가? — 인게임 시점별 경기 데이터 기반 승패 예측 및 핵심 승리요인 분석

구성: 1부 전체 구조 / 2부 13항목 상세 / 3부 모델 정리 / 4부 평가·실험 설계 / 5부 도구 사용법 / 복습 질문 / 부록 피처 명세

1부. 전체 구조

1-1. 5단계 13항목

단계	항목	완료 기준	우리 실측
1. 문제 정의	①문제번역 ②첫점검 ③EDA	목표 컬럼 y와 1순위 지표 확정	이진분류 · $y=blueWins$ · 정확도
2. 기준선	④분할 ⑤전처리설계 ⑥베이스라인	기준 점수 확보	찍기 0.50
3. 성능 개선	⑦특성공학 ⑧모델탐색	기준선보다 나은 모델	차이피쳐 13개 · 로지스틱 0.7369
4. 검증	⑨평가검증 ⑩튜닝	점수의 신뢰성 확인	5-fold 0.735 ± 0.012 (Day3 계속)
5. 산출물	⑪해석·오류분석 ⑫아티팩트 ⑬문서	인수인계 가능 상태	Day4 예정

각 항목은 앞 항목의 산출물을 입력으로 받는다. ①이 y-지표를 정해야 ②에서 볼 것이 정해지고, ⑥의 기준 점수가 있어야 ⑧의 성능 비교가 성립한다. 순서를 바꾸거나 건너뛸 수 없는 이유다.

1-2. 피드백 경로 3가지

조건	이동	이유
튜닝해도 성능이 안 오름	⑩ → ⑦	성능 상한은 하이퍼파라미터가 아니라 피쳐가 결정
점수는 오르는데 목적과 어긋남	⑨ → ①	지표 선택이 잘못됨
특정 구간에서만 실패	⑪ → ③	데이터에 파악 못 한 구조가 있음

CRISP-DM이 순환 구조인 이유. 우리 사례: 첫 군집 결과가 정답을 재현하는 무의미한 결과 → 피쳐 재설계 후 재수행 (2부 ⑦).

1-3. 필수 규칙 6

#	규칙	어기면
0	ID·상수 컬럼 먼저 제거	모델이 식별자를 암기
1	분할이 전처리보다 먼저	누수 — 오류 없이 점수만 상승
2	전처리는 Pipeline 안에	교차검증·저장·재현 불일치
3	베이스라인 먼저 측정	점수의 좋고 나쁨 판단 불가
4	학습·검증 점수 병기	과적합 진단 불가
5	Day1 끝에 점수 확보	재작업할 시간 부족

2부. 13항목 상세

형식: 무엇 / 왜 / 포인트 / 우리 프로젝트

① 문제 번역

- **무엇:** 질문 한 문장 → (문제유형, 목표컬럼 y , 1순위 지표)
- **왜:** 이 세 값이 이후 모든 결정의 입력
- **포인트:** 유형은 y 로 결정 — y 없음 → 군집 / 연속값 → 회귀 / 범주 → 분류. 지표는 오답 비용 구조로 결정(4-2). 규칙으로 풀리는 문제면 ML 불필요
- **우리:** 무승부 없음 → 이진분류. 승률 % 출력 필요 → 확률 모델. "해당 시점까지 정보만" 조건이 시간·컬럼 누수를 구조적으로 차단

② 데이터 첫 점검

- **무엇:** 크기 · 결측 · y 분포 세 가지 확인
- **왜:** 크기 → 가능한 방법 결정, 결측 → 전처리 설계, y 분포 → 지표 확정
- **포인트:** 즉시 제거 대상 = 상수 컬럼 / 문자 컬럼인데 고유값 $\approx$ 행수(ID) / y 결측 행 / 결과를 알아야 생기는 컬럼(누수). 수치 컬럼은 고유값이 많아도 자동 삭제 금지
- **우리:** 9,879×40 · 결측 0 · 균형 50.1:49.9 → 지표를 F1에서 정확도로 변경 · gameld 제거

③ EDA

- **무엇:** 모델 이전에 그림 4종 — 수치 히스토그램 / 목표 상관 순위 / 결측 패턴 / 범주 값 개수
- **포인트:** 쏠린 분포 → $\log_{10} p$ / 범위 100배 차 → 표준화 / 피쳐 간 상관 0.95 ↑ → 하나 제거 / y 와 상관 0.99 ↑ → 누수 의심, 생성 시점 확인

 상관 순위

그림①: 골드차 ± 0.51 > 경험치차 ± 0.49 > 킬 0.34 — 승리요인 1순위는 골드차. blue/red 막대가 좌우 대칭 쌍 = 동일 정보 중복(완전 중복 9쌍 — 이후 EliteMonsters 합 관계 2개를 더 찾아 총 11개 제거). y 와 0.99 이상인 컬럼 없음 — 누수 없음.

 승패 분포

그림② (골드차 그래프 기준): 가로축 = 10분 시점 골드차, 세로축 = 경기 수. 파랑 = 최종 승리한 경기들, 빨강 = 패배한 경기들. - 파란 산은 오른쪽(+), 빨간 산은 왼쪽(-)에 몰림 → 골드를 앞선 팀이 대체로 이긴다 - 단, 0 부근에서는 두 산이 포개진다(보라색) → 같은 골드차 값에서 이긴 경기와 진 경기가 섞여 있다 - 모델은 같은 입력에 답을 하나만 낼 수 있으므로, 이 구간의 경기는 어떤 모델이라도 상당수 틀린다 → 정확도가 100%가 될 수 없는 이유 - 포개지는 곳 = 골드차가 작은 경기(접전) → 오류분석 가설: "모델이 틀린 경기를 모아보면 골드차가 작은 경기가 많을 것" (Day4에 골드차 구간별 정확도로 검증)

④ 분할

- **무엇:** 훈련 80% / 테스트 20% 분리. 테스트셋은 최종 평가까지 미사용
- **왜 전처리보다 먼저:** 전체 데이터로 전처리하면 테스트셋 통계가 학습에 반영됨(누수). 누수는 오류 없이 점수만 올라 코드상 발견되지 않음
- **포인트:** 일반 분류 `stratify=y` / 회귀 단순 분할 / 시계열은 시간순 분할 필수 / 동일 개체 다중 행이면 `GroupShuffleSplit`
- **우리:** 층화 8:2, `random_state=42`, 인덱스 파일 저장 → 모든 실험이 동일 분할 사용

⑤ 전처리 설계

- **무엇:** 학습셋 `fit_transform` / 테스트셋 `transform`. 이 구분을 Pipeline이 자동 강제
- **왜:** 교차검증의 각 fold에서 전처리가 해당 fold 학습부로만 fit되도록 보장. 저장 시 전처리+모델이 한 객체로 유지
- **포인트:** `ColumnTransformer`는 미지정 컬럼을 경고 없이 제거 — `assert`로 확인
- **우리:** `Pipeline([스케일러, 모델])`. 결측 0이라 대치 단계 없음

⑥ 베이스라인

- **무엇:** 무학습 모델 점수 — 분류 `DummyClassifier(most_frequent)` / 회귀 `DummyRegressor(mean)`
- **왜:** 비교 기준 없이는 점수 해석 불가
- **포인트:** 개선의 대부분은 첫 단순 모델에서 나온다(가이드라인 실측: 선형회귀가 전체 개선폭의 76%). 지표별로 적합한 베이스라인이 다름 — `most_frequent`는 양성 미예측으로 $F1=0$
- **우리:** 정확도 기준선 0.50(`most_frequent`) / F1 기준선 0.50(`stratified`)

⑦ 특성 공학

- **무엇:** 원본 컬럼을 문제에 맞는 표현으로 변환. 성능 상한을 올리는 주 수단
- **포인트:** 좋은 특성 4조건 — 목표와 관련 / 예측 시점에 존재(누수 체크) / 중복 없음 / 값 안정. 유형 — 비율, 구간화(`pd.cut`), 집계(학습셋 기준만), 로그(`log1p`), 차이
- **우리:** 중복 11개 제거 후 블루-레드 차이 13개로 재구성. 동일 분할·모델에서 피처만 교체해 비교: 27개 0.7334 vs 13개 0.7369 → 채택. PCA 압축은 해석 불가로 기각

비지도 분석 (우리 프로젝트 추가 수행)

 PCA와 k=2 군집

그림③: (좌) PC1 37% = 골드·경험치·킬·레벨 동반 이동(성장 격차 축), PC2 12% = 드래곤·전령(오브젝트 축). (우) 차이 피처로 군집하면 k=2가 PC1=0 기준으로 갈라짐 = 블루 우세/레드 우세(승률 73/27%). 정답과 동일한 정보라 분석 가치 없음 → 재설계.

 경기 4유형

그림④: 진영 부호를 제거한 피쳐(|골드차|·총킬·총오브젝트·총와드·총CS)로 재군집한 결과.

유형	비중	특징	10분 골드 리드팀의 최종 승률
운영전 (빨강·좌하)	42%	킬 9, 교전 적음	68%
난타전 (하늘·좌상)	32%	킬 15, 교전 많음	68%
시야전 (그림에 미표시)	7%	와드 120 (평균의 3배)	70%
일방적 (분홍·우측)	19%	골드차 4,200	90%

골드차 4,000 이상이면 10분 시점에 승부가 사실상 확정(90%), 접전은 68%. 시야전이 그림에 없는 이유: 해당 유형을 가르는 피쳐(와드)가 그림의 두 축에 없음. k 선택: 실루엣이 k=2~6에서 0.203~0.219로 전부 비슷해(폭 0.016) k를 정해주지 못한다. 최고는 k=3(0.219)이고 k=4(0.211)는 k=5(0.214)에도 밀린 3위 — 점수가 아니라 해석 용이성으로 k=4 선택. 값 자체가 낮아 이 데이터에 뚜렷이 분리된 덩어리는 없으며, 우리 군집은 '설명을 위한 구획'에 가깝다(한계로 명시).

※ 위 승률 수치는 군집별로 데이터를 세어본 **기술 통계**이지 모델의 예측이 아니다. ML 전용 구현 제약 (예측·해석은 학습된 모델에서만 산출)에 따라, 최종 관전 가이드에 실을 수치는 Day4에 학습 모델의 predict_proba로 재산출한다. "골드차>4천 → 승리" 같은 규칙은 예측 로직에 쓰지 않는다.

⑧ 모델 탐색

- **무엇:** 후보 모델별 cv 평균 · 표준편차 · 소요시간 비교표 작성
- **순서:** ①로지스틱/선형(빠르고 해석 가능, 새 기준선) → ②랜덤포레스트 → ③HistGradientBoosting(대체로 최고 성능) → ④KNN·SVM(선택, 스케일 필수) → ⑤Voting(모델 간 다양성 있을 때만)
- **포인트:** 성능이 안 오를 때 순서 = 모델 교체 → 특성 공학 → 튜닝. 튜닝을 먼저 하면 개선폭이 가장 작음
- **우리:** Day3 예정. 로지스틱 0.735가 비교 기준

⑨ 평가·검증

- **무엇:** 혼동행렬 기반 지표(4부) + 임계값 근거. 회귀는 MAE·RMSE·R²+잔차 그림, 군집은 실루엣+프로파일 표
- **포인트:** predict_proba 로 확률을 받아 임계값 0.3~0.7 확인. 0.5는 기본값일 뿐. 재현율이 부족하면 모델 교체 전에 임계값 조정이 우선(추가 비용 없음)
- **우리:** 비용 대칭이므로 최적 임계값 0.5 예상 — 확인 자체가 결과. Day3 예정

⑩ 튜닝

- **무엇:** 확정된 피쳐·모델에서 하이퍼파라미터 최적화. 순서상 마지막
- **포인트:** 학습 횟수 = 조합 수 × cv (예: 12조합 × cv5 = 60회). 50조합 미만 GridSearchCV, 이상 RandomizedSearchCV. 개선이 없으면 ⑦로 복귀

- **문법:** Pipeline 사용 시 파라미터명은 "단계이름__파라미터"

⑪ 해석·오류 분석

- **무엇:** 예측 근거와 오분류 패턴 도출. 배점 10점 — "성능 0.82"보다 "어떤 경우에 틀리는지"가 평가 대상

도구	측정 대상	주의
<code>feature_importances_</code>	학습 시 분할 기여	고유값 많은 컬럼 과대평가
<code>permutation_importance</code>	테스트셋에서 피처를 섞었을 때 점수 하락	느리지만 신뢰도 높음
회귀 계수	피처 1 증가 시 y 변화	단위가 다르면 비교 불가(표준화 후 비교)

- 상관이 강한 피처 간에는 세 도구 모두 중요도가 분산되어 왜곡됨 → 중복 제거를 선행하는 이유
- **오류 분석 방법:** 오분류 표본의 피처 평균 - 전체 평균 = 오분류 공통점
- **우리:** 계수(방향)와 permutation(크기) 양쪽 상위인 것만 승리요인으로 인정. 가설 "골드차가 작을 수록 오분류"를 구간별 정확도로 검증 예정

⑫ 아티팩트

- **무엇:** 전처리 포함 Pipeline 전체를 joblib으로 저장, 복원 후 예측 일치를 assert로 확인
- **재현성 규약 7:** 시드 고정 / 분할 파일 저장 / 절대경로 금지 / requirements.txt 버전 고정 / 노트북 순차 실행 / 전처리 Pipeline 내부 / 실험 즉시 runs.csv 기록

⑬ 문서·발표

- **산출물 6종:** README / 무오류 최종 노트북 / model.joblib / schema.json(입력 컬럼·타입·허용범위) / predict.py / model_card.md(용도·성능·사용 금지 상황)
- **포인트:** 배점 = 성능 12 vs 재현성·해석·문서 28 + 오류분석 10. model_card에 유효 입력 범위를 명시 — 학습 범위 밖 입력(외삽)에서 트리는 끝값 고정, 선형은 발산으로 서로 다르게 틀림

3부. 모델 정리

3-1. 비교표

모델	원리	스케일링	해석성	비고
Dummy	무학습 예측	불필요	—	기준선 전용
로지스틱 회귀	가중합 → 시그모이드로 확률 변환	필요	상 (계수)	빠름, 확률 출력. 비선형 관계 못 잡음
결정트리	불순도 최소 기준으로 조건 분할	불필요	상 (경로)	단독 사용 시 과적합
랜덤포레스트	다수 트리의 다수결 (배깅)	불필요	중	튜닝 없이도 안정적. 느림
HistGradientBoosting	이전 트리의 잔차를 다음 트리가 학습 (부스팅)	불필요	중	대체로 최고 성능, 빠름. 튜닝 민감
KNN	최근접 k개 이웃의 다수결	필수	하	예측 느림, 고차원에 약함
SVM	클래스 간 마진 최대화	필수	하	고차원에 강함, 느림
Voting	복수 모델의 표결	구성원 따름	하	모델 간 다양성이 전제
KMeans	중심점 k개 기준 거리 군집	필수	프로파일로	k는 사용자가 결정
PCA	분산 최대 방향으로 축 변환·압축	필수	적재량으로	주성분은 원 피쳐 의미를 잃음

3-1b. 우리 데이터 실측 — 통념이 깨진 사례 (Day 3)

위 3-1 표는 **일반론**이다. 실제로 8종을 같은 조건(29피쳐, 5-fold, seed 42, 지표=정확도)에 붙여본 결과는 아래와 같다.

순위	모델	교차검증	학습 점수	과적합(학습-검증)
1	로지스틱 회귀	0.7280 ± 0.0163	0.7319	+0.0039
2	랜덤포레스트	0.7248 ± 0.0151	1.0000	+0.2752
3	GaussianNB	0.7238 ± 0.0172	0.7253	+0.0015
4	SVM (RBF)	0.7223 ± 0.0170	0.7614	+0.0391
5	결정트리(depth=4)	0.7180 ± 0.0140	0.7285	+0.0105
6	KNN(k=25)	0.7147 ± 0.0133	0.7353	+0.0206
7	HistGradientBoosting	0.7140 ± 0.0153	0.8527	+0.1387
—	Dummy	0.5009 ± 0.0002	0.5009	0.0000

배울 점 3가지

1. **"HistGB가 대체로 최고"는 통념이지 법칙이 아니다.** 2부 ⑧의 탐색 순서에 그렇게 적혀 있지만, 우리 데이터에서는 **꼴찌**였다. 순서는 '무엇부터 시험할지'이지 '무엇이 이길지'가 아니다.
2. **학습 점수를 같이 봐야 진짜가 보인다.** 랜덤포레스트는 검증 0.7248 로 2위지만 학습이 **1.0000** 이다 — 문제집을 통째로 외웠다. 검증 점수만 봤다면 "관창은 2위"로 착각했을 것이다. 필수 규칙 4 번(학습-검증 병기)이 존재하는 이유가 이것이다.
3. **성능이 복잡도의 역순인 경우가 있다.** 피처가 적고 관계가 대체로 선형이면 복잡한 모델은 잡음까지 학습한다. 실제로 blueGoldDiff 1개(0.7313) + blueExperienceDiff 2개(0.7328)가 29개 전체 (0.7303)를 이겼다 → **성능 상한은 모델이 아니라 피처가 결정한다**(1-2 피드백 경로 ⑩→⑦).

우리가 고른 것: 로지스틱 회귀. 정확도 1위 + 과적합 거의 없음 + 계수로 설명 가능. 프로젝트의 핵심 가치가 "왜까지 설명"이므로, 점수가 비슷하면 설명되는 모델이 맞다.

3-2. 원리에서 유도되는 성질

1. **양상불:** 배깅(RF)은 서로 다르게 틀리는 트리들의 평균으로 분산을 줄인다 → 튜닝 없이 안정적. 부스팅(HistGB)은 잔차를 순차 보정해 편향을 줄인다 → 튜닝하면 강하고, 과하면 과적합
2. **스케일링 필요 여부:** 가중합·거리·마진·분산 등 값의 크기를 계산에 쓰면 필요(로지스틱·KNN·SVM·KMeans·PCA), 임계값 비교만 하면 불필요(트리 계열)
3. **모델 선택 기준:** 해석이 목적이면 로지스틱 중심, 성능 상한 확인은 트리 양상불, 확률 출력이 필요하면 predict_proba 지원 모델

3-3. PCA(주성분 분석) 상세

풀려는 문제. 피처가 13개면 데이터는 13차원 공간에 있다. 사람은 2~3차원까지만 볼 수 있고, 피처끼리 정보가 겹치는 경우도 많다. 차원을 줄이면서 정보를 최대한 보존하는 방법이 필요하다.

원리. 데이터가 가장 넓게 퍼진 방향(분산이 최대인 방향)이 정보가 가장 많은 방향이다. 1. 그 방향을 첫 번째 새 축(PC1)으로 잡는다 2. PC1과 직각인 방향 중 분산이 최대인 방향을 PC2로 잡는다 3. 반복하면 피쳐 수만큼 새 축이 나온다

이 단계까지는 좌표축을 **회전만** 한 것이라 정보 손실이 없다. 압축은 그 다음이다 — 뒤쪽 축들은 분산(정보)이 거의 없으므로, 앞쪽 몇 개만 남기고 버리면 적은 손실로 차원이 줄어든다.

비유 하나: 학생들의 국어·영어·사회 점수가 항상 같이 오르내린다면, 세 과목을 따로 볼 필요 없이 "언어 능력"이라는 축 하나로 대부분 설명된다. PCA는 이런 "같이 움직이는 피쳐 묶음"을 자동으로 찾아 축으로 만든다.

읽는 법 — 산출물 3가지.

산출물	코드	의미
설명 분산 비율	<code>explained_variance_ratio_</code>	각 축이 전체 정보의 몇 %를 담는가
누적 비율	<code>np.cumsum(...)</code>	몇 개 축이면 90%가 되는가 → 압축 시 유지할 축 수 결정
적재량	<code>components_</code>	각 축이 원래 피쳐들의 어떤 조합인가 → 축의 의미 해석

적재량 절대값이 큰 피쳐들을 보고 축에 의미를 부여한다. 우리 실측(차이 피쳐 13개): PC1(38.6%)의 적재 상위가 GoldDiff·ExpDiff·KillsDiff·AvgLevelDiff·AssistsDiff·TotalMinionsKilledDiff → 돈·경험치·킬·레벨이 함께 움직이는 "성장 격차 축"이 데이터의 약 40%를 설명한다. PC2는 9.3%에 그쳐 두 번째 이야기는 훨씬 약하다. ※ 입력을 바꾸면 축도 바뀐다 — 사이드 중립 5피쳐로 돌리면 PC1 32.0%(교전↔성장), PC2 20.4%(시야↔오브젝트)로 전혀 다른 축이 나온다.

표준화가 필수인 이유. PCA는 분산을 기준으로 축을 잡는다. 표준화 없이 돌리면 단위가 큰 피쳐(골드, 수천 단위)의 분산이 압도적이어서 PC1이 사실상 골드 축이 되어버린다. 모든 피쳐를 평균 0·표준편차 1로 맞추는 뒤 적용해야 공정한 비교가 된다. fit은 학습셋으로만(누수 방지).

용도 3가지와 한계.

용도	설명	우리 프로젝트
시각화	고차원 데이터를 PC1·PC2 평면에 투영	군집 결과를 2D로 확인 (그림③ 우측)
구조 해석	적재량으로 "같이 움직이는 피쳐 묶음" 파악	우세 = 성장 축 + 오브젝트 축 발견
압축	앞쪽 축만 남겨 피쳐 수 축소, 다중공선성 완화	기각 — 아래 한계 때문

한계: ① 주성분은 원래 피쳐들의 혼합이라 **이름이 없다** — "PC1이 0.3 증가하면 승률이 오른다"는 설명이 되지 않으므로, 해석이 목적인 모델의 입력으로는 부적합 (우리가 압축 용도를 기각한 이유). ② 직선적(선형) 조합만 잡는다 — 곡선 관계는 놓친다. ③ 압축한 만큼 정보 손실이 있다 (우리 데이터는 누적 90%에 8축 필요(13축 중) → 중복이 아주 심하지는 않다는 뜻이기도 함).

군집과의 관계. PCA와 KMeans는 둘 다 비지도 학습이지만 역할이 다르다 — PCA는 축을 바꾸는 것(요약), KMeans는 표본을 묶는 것(분류). 우리 프로젝트에서는 PCA 평면을 군집 결과의 시각화 좌표로 썼고, 군집 입력 자체는 별도 설계한 사이드 중립 피쳐를 사용했다.

4부. 평가·실험 설계

4-1. 혼동행렬과 파생 지표

	예측 음성	예측 양성
실제 음성	TN	FP
실제 양성	FN	TP

지표	식	사용 조건
정확도	$(TP+TN)/\text{전체}$	균형 + 오답 비용 대칭
정밀도	$TP/(TP+FP)$	FP 비용이 클 때 (예: 스팸 오판)
재현율	$TP/(TP+FN)$	FN 비용이 클 때 (예: 질병 미탐지)
F1	정밀·재현의 조화평균	불균형 데이터

4-2. 지표 선택 절차

y가 균형인가?
└ 불균형 → 정확도 착시 발생 (다수 클래스로만 찍어도 높음) → F1/정밀도/재현율
└ ┌ FN이 비싸면 재현율, FP가 비싸면 정밀도
└ 균형 → 오답 비용이 대칭인가?
└ ┌ 대칭 → 정확도. 최적 임계값 0.5 예상 ← 우리 경우
└ ┌ 비대칭 → 비용이 큰 쪽의 지표

4-3. 과적합 진단

학습 점수	검증 점수	진단	대응
낮음	낮음	과소적합 (편향 큼)	더 복잡한 모델, 더 나은 피처
높음	낮음	과적합 (분산 큼)	규제, 단순화, 데이터 추가
높음	높음 (격차 작음)	정상	진행

우리 실측: 학습 0.737 / 검증 0.735, 격차 0.002 — 정상.

4-4. 교차검증

보고 단위는 평균 · 표준편차 · 학습-검증 격차 세 값. 표준편차가 크면 fold 간 점수가 불안정하다는 뜻. 종류는 분할 상황표와 대응 — StratifiedKFold(분류 기본) / GroupKFold(동일 개체 분리) / TimeSeriesSplit(시계열).

4-5. 누수 5종

유형	예	차단
전처리 누수	전체 데이터로 스케일러 fit 후 분할	분할 먼저 + Pipeline
컬럼 누수	결과 이후 생성되는 컬럼 사용	생성 시점 확인
시간 누수	미래 데이터로 과거 예측	시간순 분할
중복 누수	동일 개체가 train/test 양쪽에	GroupKFold
타깃 누수	타깃으로 만든 파생 피쳐	파생 로직 검토

공통 신호: 점수가 비정상적으로 높으면 원인 확인부터. y와 상관 0.99 이상 컬럼은 누수 의심.

4-6. 비교 실험 설계

- **교란변수**: 비교하려는 변수 외에 다른 조건도 함께 바뀌면 원인 특정 불가. 우리 사례 — 10분(다이아 랭크) vs 15분(프로 경기) 비교는 시점 효과와 실력대 효과가 섞임 → 동일 경기 집합 내 비교로 수정
 - **표본 선택 편향**: 표본 포함 기준이 결과와 관련되면 왜곡. 우리 사례 — 15분 전 종료 경기(일방적 경기)가 한쪽에만 포함되면 안 됨 → 15분 기록이 있는 경기만 남기고 두 시점 모두 평가
 - **원칙**: 비교 변수 하나만 바꾸고 나머지(데이터·분할·전처리·검증)는 고정. 피쳐 세트 비교(30 vs 14)도 동일 원칙 적용
-

5부. 도구 사용법

train_test_split — 분할 (④)

훈련/테스트 분리. 전처리보다 먼저, 한 번만 수행.

```
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, stratify=y, random_state=42)
```

- `test_size=0.2`: 20%를 테스트로 / `stratify=y`: 양쪽 y 비율 유지 / `random_state`: 분할 고정
- 시계열에는 사용 금지(시간순 분할). 분할 인덱스는 파일로 저장해 재사용

Pipeline + StandardScaler — 전처리 (⑤)

표준화(평균0·표준편차1)와 모델을 한 객체로 묶는다. 교차검증·저장 시 누수 차단.

```
pipe = Pipeline([("scaler", StandardScaler()), ("model", LogisticRegression(max_iter=1000))])
pipe.fit(X_tr, y_tr)      # 스케일러는 X_tr 기준으로만 fit
pipe.predict(X_te)       # 테스트에는 transform만 적용됨
```

- 단계 이름("scaler", "model")은 튜닝 파라미터명에 사용됨
- Pipeline 밖에서 전체 데이터로 fit하면 전처리 누수

DummyClassifier — 베이스라인 (⑥)

```
DummyClassifier(strategy="most_frequent") # 다수 클래스로만 예측 → 정확도 기준선
DummyClassifier(strategy="stratified")    # 비율대로 무작위 예측 → F1 기준선
```

- `most_frequent`는 양성을 예측하지 않아 F1=0. 지표에 맞는 strategy 선택

cross_validate — 교차검증 (⑨)

```
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
res = cross_validate(pipe, X_tr, y_tr, cv=cv, scoring=["accuracy", "f1"], return_train_score=True)
res["test_accuracy"].mean(), res["test_accuracy"].std()
```

- `return_train_score=True`: 과적합 진단용 학습 점수 포함 / `scoring=[...]`: 복수 지표 동시 측정
- 입력은 훈련셋만. 테스트셋은 최종 1회 평가용

predict_proba + 임계값 (⑨)

```
prob = pipe.predict_proba(X)[: , 1]      # [: , 1] = 양성 클래스 확률
pred = (prob >= t).astype(int)           # t를 0.3~0.7 범위에서 확인
```

GridSearchCV — 튜닝 (⑩)

```
grid = {"model__C": [0.1, 1, 10]}        # "단계이름__파라미터"
gs = GridSearchCV(pipe, grid, cv=cv, scoring="accuracy", n_jobs=1)
gs.fit(X_tr, y_tr); gs.best_params_; gs.best_estimator_
```

- 실행 전 학습 횟수(조합 수 × cv) 계산. 50조합 이상이면 RandomizedSearchCV

계수 / permutation_importance — 해석 (⑪)

```
pipe["model"].coef_[0]                  # 부호=방향, 크기=영향(표준화 입력 기준)
from sklearn.inspection import permutation_importance
r = permutation_importance(pipe, X_te, y_te, n_repeats=10, random_state=42)
r.importances_mean                      # 피처를 섞었을 때 점수 하락폭
```

- 상관 강한 피처 간에는 양쪽 다 왜곡 → 중복 제거 후 해석

PCA — 구조 요약

```
Z = StandardScaler().fit_transform(X_tr)  # 분산 기반이므로 표준화 필수
pca = PCA().fit(Z)
pca.explained_variance_ratio_             # 축별 설명 비율
pca.components_                          # 적재량: 축의 구성 피처 확인
```

- 주성분은 원 피처의 의미를 잃으므로 해석이 필요한 모델의 입력으로는 부적합

KMeans + silhouette_score — 군집

```
km = KMeans(n_clusters=k, n_init=10, random_state=42).fit(Z)
silhouette_score(Z, km.labels_)           # k 후보 비교 (1에 가까울수록 좋음)
df.groupby(km.labels_).mean()             # 군집 프로파일
```

- 군집 결과가 정답 변수를 재현하면 분석 가치 없음 — 입력 피처 설계가 먼저
- k 최종 선택은 실루엣 + 해석 용이성

joblib — 저장 (⑫)

```
joblib.dump(pipe, "artifacts/model.joblib")
restored = joblib.load("artifacts/model.joblib")
assert (restored.predict(X_te) == pipe.predict(X_te)).all() # 복원 검증까지 수행
```

pymysql — DB 적재

```
conn = pymysql.connect(host=..., port=..., user=..., password=..., database=...)
cur.execute("CREATE TABLE ... (gameId BIGINT, ..., PRIMARY KEY (gameId))")
cur.executemany(insert_sql, rows[i:i+1000]) # 1,000행 단위 배치
conn.commit() # 누락 시 저장 안 됨
```

- 타입은 의미대로(ID=BIGINT, 0/1=TINYINT, 소수=DOUBLE), PK로 중복 차단
 - 적재 검증: 행 수 / 대표 합계 / 대표 평균을 원본과 대조
 - 비밀번호는 하드코딩하지 않고 실행 인자로 전달
-

복습 질문

1. ⑥베이스라인이 ⑦특성공학보다 앞인 이유는? — [2부⑥⑦]
 2. 피드백 경로 3가지의 조건은? 우리가 실제로 수행한 경로는? — [1-2]
 3. y분포 실측이 지표 선택을 바꾼 과정을 4-2 절차로 설명하라
 4. 그림①에서 중복 쌍을 확인하는 방법과, 중복이 해석을 왜곡하는 원리는? — [2부③⑪]
 5. 그림②의 분포 겹침이 정확도 상한을 만드는 이유는? — [2부③]
 6. 누수 5종을 예와 함께 설명하고, 우리 문제 설정이 그중 둘을 차단하는 이유는? — [4-5, 2부①]
 7. 배경과 부스팅이 각각 줄이는 것은? 그로 인한 성격 차이는? — [3-2]
 8. 스케일링 필요 여부의 판별 기준은? — [3-2]
 9. 균형+대칭 비용에서 최적 임계값이 0.5인 이유와 검증 방법은? — [4-2, 2부⑨]
 10. 시점 비교 실험의 교란변수·선택편향과 통제 방법은? — [4-6]
 11. 첫 군집이 무의미했던 이유(그림③ 우측)와 재설계 방법은? — [2부⑦]
 12. permutation_importance의 장점과, 중요도 도구들이 공통으로 왜곡되는 조건은? — [2부⑪]
 13. 산출물 6종과 model_card에 유효 범위를 적는 이유(외삽)는? — [2부⑬]
 14. 학습 0.737/검증 0.735에서 확인되는 사실 두 가지는? — [4-3]
-

부록 A. 피처 명세

데이터: data/high_diamond_ranked_10min.csv (9,879경기 × 40컬럼). 모든 수치는 10분 시점까지의 누적 값. blueoo / redoo 쌍으로 19항목 × 2팀 + gameId + blueWins.

A-1. 원본 컬럼

컬럼	설명	처리/비고
gameId	경기 고유 번호	제거 — 식별자
blueWins	정답 y: 블루 승=1, 패=0	목표 컬럼
WardsPlaced	와드(시야 확보 아이템) 설치 수	
WardsDestroyed	상대 와드 제거 수	
FirstBlood	첫 킬 여부 (0/1)	blue+red=1 → red 제거
Kills / Deaths	킬 / 데스	blueKills=redDeaths → red 쪽 제거
Assists	어시스트	
EliteMonsters	대형 몬스터 합	= Dragons+Heralds
Dragons / Heralds	드래곤 / 전령 처치 수	10분 시점 보통 0~1
TowersDestroyed	타워 파괴 수	10분 시점 대부분 0
TotalGold	팀 골드 합계	
AvgLevel / TotalExperience	평균 레벨 / 경험치 합계	상호 강한 상관
TotalMinionsKilled	미니언(CS) 수	
TotalJungleMinionsKilled	정글 몬스터 수	
GoldDiff / ExperienceDiff	골드/경험치 차이 (자기 팀 - 상대)	blue=-red → red 제거
CSPerMin / GoldPerMin	분당 미니언/골드	= 총량÷10 → 제거

정리 결과: 38피처 → 1차 중복 9개(상관 ±1.0) + 2차 2개(EliteMonsters = Dragons + Heralds) 제거 → 27개

A-2. 모델 입력 — 차이 피처 13개

27개 0.7334 → 13개 0.7369로 채택. 양수 = 블루 우세.

피처	정의
FirstBlood	blueFirstBlood
KillsDiff	blueKills - redKills
GoldDiff	원본 그대로 (상관 1위 0.51)
ExpDiff	원본 그대로 (상관 2위 0.49)
WardsPlacedDiff WardsDestroyedDiff AssistsDiff DragonsDiff HeraldsDiff TowersDestroyedDiff AvgLevelDiff TotalMinionsKilledDiff TotalJungleMinionsKilledDiff	각 blue - red

A-3. 군집용 사이드 중립 피처 5개

피처	정의
일방성_골드차	blueGoldDiff
난타전_총킬	양 팀 킬 합
오브젝트_총획득	양 팀 드래곤+전령 합
시야전_총와드	양 팀 와드 합
성장_총CS	양 팀 미니언 합

A-4. PCA 축

축	분산	주요 적재	해석
PC1	38.6%	GoldDiff·ExpDiff·KillsDiff·AvgLevelDiff·AssistsDiff·TotalMinionsKilledDiff	성장 격차 축
PC2	9.3%	HeraldsDiff·TotalJungleMinionsKilledDiff·TowersDestroyedDiff	정글·오브젝트 성격

※ 차이 피처 13개 기준. 사이드 중립 5피처로 돌리면 PC1 32.0%(교전↔성장) / PC2 20.4%(시야↔오브젝트)로 축이 달라진다.